

15-418 Assignment 3: Parallel VLSI Wire Routing Report

Jason (jasont2) Catherine (catheri5)

October x, 2026

1 Implementation Status

We first implemented a sequential correctness baseline before adding OpenMP parallelism. The starter code's compressed route representation was replaced with a keypoint representation: each wire stores up to five points, corresponding to the start point, up to three bends, and the end point. This matches the assignment checker format and makes route construction and validation easier to reason about.

At the start of timed computation, each input wire is assigned a simple legal route. If the endpoints share a row or column, the initial route is straight. Otherwise, the initial route is a horizontal-first one-bend path. The occupancy matrix is then constructed by adding each initial wire route.

The current baseline rerouting algorithm visits every wire for each iteration. For a wire being reconsidered, the current route is first removed from the occupancy matrix. With fixed probability P , the program chooses a legal route uniformly at random. Otherwise, it evaluates every legal route with at most three bends using the sum of squared occupancy values that would result from adding the wire back, and chooses the lowest-cost route. The existing route is preserved unless a strictly cheaper candidate is found. The selected route is then added back into the occupancy matrix.

2 Route Generation and Cost Calculation

For non-collinear endpoints, route generation currently includes horizontal-first and vertical-first routes with at most two bends, plus exactly three-bend routes through each strictly interior point in the endpoint bounding box. Choosing strictly interior points for the three-bend case prevents those

routes from collapsing into duplicate one- or two-bend routes. Some at-most-two-bend routes do collapse to one-bend routes; consecutive duplicate keypoints are removed when constructing the wire.

When scoring a candidate route, the current wire has already been factored out of the occupancy matrix. Therefore each occupied cell on the candidate route is scored using $(occupancy[y][x] + 1)^2$. This computes the route’s relevant contribution if the current wire were added back along that path.

3 Sequential Baseline Optimization

Table 1 summarizes the historical computation checkpoints on GHC for `few.wires.txt`, one thread, $P = 0.1$, five iterations, and batch size one. The early W-mode measurement provides context; the subsequent optimizations were first tested in the sequential A-mode baseline before being ported to W.

Implementation checkpoint	Mode	Compute (s)
Early vector-based candidate search	W	6.14
Generate and score immediately, without a candidate vector	A	≈ 5.89
Also specialize horizontal/vertical cost loops	A	3.17
Also maintain a contiguous column occupancy view	A	≈ 2.68

Table 1: Exploratory baseline progression, with times shown to three significant figures. The 5.89 s and 2.68 s values were reported approximately. The 3.17 s value is a representative run, with a reported median of approximately 3.15 s.

The recorded stages matter: eliminating the route vector alone left the A-mode baseline near 5.89 seconds, rather than immediately reaching 3.17. That larger reduction followed the axis-specific scoring loop; contiguous vertical access then reduced the reported time to about 2.68 seconds. From the 5.89-second A-mode checkpoint to 2.68 seconds, computation time fell by approximately 54.5% (about $2.20\times$ speedup). These historical checkpoints were not all collected as a controlled median-of-three comparison, and W and A use different search paths, so we treat them as evidence of progress rather than a final benchmark comparison.

Before further within-wires tuning, we focused on the efficiency of the

one-thread baseline. Originally, we stored all candidate routes in a vector before scoring them. We changed the sequential search to construct and fully score each candidate immediately, keeping only the current candidate and best route. This removes candidate-vector allocation, growth, and storage overhead without changing the exhaustive route set. The random branch constructs just the uniformly selected route without building the candidate list.

We also specialized the cost loop for horizontal and vertical segments. Horizontal scoring fixes a row and changes only the column index; vertical scoring fixes a column and changes only the row index. Initially, vertical access still crossed separately allocated rows. We subsequently added a transposed raw occupancy view, `occupancy_columns[x][y]`, alongside `occupancy[y][x]`. Horizontal segments now read contiguous rows and vertical segments read contiguous columns. Every wire removal and insertion updates both views. This trades additional memory and update work for better locality during the many candidate-scoring reads.

The transposed view is not a prefix-sum table or a cache of route costs. Every candidate is still evaluated completely, without cost reuse or early termination, and bends and the final endpoint are counted exactly once. The optimized traversal is now used by both modes. A mode generates candidates directly in sequential loops; W mode uses indexed generation as described below.

4 Within-Wires Parallelism

We keep iterations and the outer loop over wires sequential. After removing the current wire from both occupancy views, all candidates for that wire can be evaluated independently against the same read-only occupancy state. The chosen route is committed before moving to the next wire.

Our initial parallel implementation generated a candidate vector sequentially and parallelized only scoring. The current implementation instead maps an integer route index to one candidate with `route_from_index()`. Each worker constructs and scores its assigned candidates immediately, so candidate generation is also parallel and the full route vector is no longer stored. The exact candidate count is available without constructing routes: for non-collinear endpoints it is

$$N_{\text{routes}} = d_x + d_y + 2(d_x - 1)(d_y - 1),$$

where d_x and d_y are absolute endpoint differences. Collinear endpoints have

one candidate. This preserves a route-count-based sequential fallback.

Each thread maintains a private best route and cost. At the end of the parallel search, the main thread combines the per-thread results. This is a manual minimum reduction, analogous to accumulating partial sums independently and combining them once, rather than synchronizing after every candidate.

4.1 Tuning Methodology and Decisions

For early within-wires tuning, we temporarily measured only the iterative rerouting phase, excluding initial route placement and occupancy initialization. This narrower timing made it easier to isolate the effect of scheduling, chunk size, and threshold choices. Those older implementation measurements are not used for the current choices below. Current measurements use the assignment timing definition, where computation includes initial route placement and occupancy filling.

For the current sweeps, we used `medium.wires.txt`, eight threads, $P = 0.1$, five iterations, and batch size one. Each computation time is the median of three runs per configuration, with one knob varied at a time. All supplied sweep outputs passed validation and had maximum occupancy three.

Table 2 collects the three sweeps. Bold settings are the selected choices; differences within each sweep were small.

Sweep	Setting	Compute median (s)	Cost
Schedule	static	1.504505	267645
	dynamic	1.535503	267681
Chunk size	1	1.486193	267735
	4	1.513441	267641
	8	1.509649	267645
	16	1.499131	267423
	32	1.502043	267425
Threshold multiplier	2	1.517222	267735
	4	1.486254	267735
	8	1.488452	267735
	16	1.489102	267735
	32	1.506372	267735

Table 2: Medium-input tuning on GHC at eight threads. Schedule sweep: chunk 8, multiplier 4. Chunk sweep: static, multiplier 4. Threshold sweep: static, chunk 1. All supplied outputs passed validation.

We kept static scheduling, which used about 2% less computation time than dynamic. Static avoids dynamic work-dispatch overhead, but this modest difference does not isolate overhead as the cause. Chunk one had the lowest median, about 1.6% below chunk eight; the full chunk sweep spanned only about 1.8%. These results support a provisional configuration, not a claim that scheduling or chunk size dominates performance.

Fine granularity. In `schedule(static, 1)`, a chunk is one candidate route, not one cell. Candidates are assigned round-robin without a dynamic task queue. A candidate still performs a complete path evaluation, so a small chunk does not imply that each scheduled unit contains only a trivial amount of work.

Importantly, all generated candidates for a particular wire have the same Manhattan length, $d_x + d_y$, and visit the same number of cells. Path lengths vary between wires, but the W-mode parallel loop processes only one wire’s candidates at a time. Therefore, the chunk-one result cannot be explained by balancing different wire lengths. Candidate orientations, bend counts, and memory access patterns can differ even with equal path length; a fine cyclic assignment may distribute those differences, but these effects were not separately measured. The timing data justify choosing chunk one provisionally, not a general claim that finer granularity is always better.

The parallel threshold is the multiplier times the thread count: the tested values correspond to 16, 32, 64, 128, and 256 candidates at eight threads. We retained multiplier four; the corrected medians for 4, 8, and 16 differed by only about 0.2%. The sequential fallback avoids entering a parallel region for very small searches, but this sweep does not establish an exact optimal cutoff or how frequently each threshold changes the execution path.

Our selected configuration is therefore static scheduling, chunk size one, and threshold multiplier four. These decisions are based on medium-input measurements and must still be checked on few and abundant inputs and in the final scaling evaluation.

4.2 Synchronization and Shared Memory

Threads share both occupancy views, but only read them during candidate scoring, so no occupancy locks are needed in W mode. Multiple threads reading the same cache line do not cause write-induced coherence invalidations and may benefit from shared cache reuse. This is distinct from false sharing, which requires writes to a shared cache line; our local bests remain private throughout scoring. Actual cache benefits depend on working-set

size and scheduling and have not yet been quantified with cache-miss measurements.

The worksharing loop has an implicit barrier. Each worker then publishes its local best to its own result slot, and the parallel-region end synchronizes completion before the main thread reduces those results. Occupancy removal and insertion remain sequential, outside the parallel region. Thus no critical section or atomic update is needed for each candidate comparison.

We moved the per-thread result-vector allocations outside the iteration and wire loops, reusing storage across searches. Before each parallel search, cost slots reset to the current wire’s starting best; active threads overwrite their route slots. Unused slots cannot select stale routes because the final reduction accepts only strictly cheaper costs. Buffer reuse showed no measurable computation-time improvement, so we retain it as a simple allocation cleanup rather than attributing a speedup to it.

Equal-cost route choices can differ across thread counts and schedules because the reduction does not impose a deterministic candidate-index tie-break. Those choices can affect later wires and explain different final costs without an occupancy mismatch. Consequently, some tuning runs follow different routing trajectories rather than exactly identical work.

4.3 Scaling Limits

Table 3 shows the initial current-implementation medium snapshot, using static chunk eight and multiplier four. This is a preliminary checkpoint, not the final evaluation of the selected chunk-one configuration.

Threads	Compute (s)	Compute speedup	Total speedup	Efficiency
1	9.511255	1.00×	1.00×	100.0%
2	4.941826	1.92×	1.92×	96.2%
4	2.664504	3.57×	3.53×	89.2%
8	1.506321	6.31×	6.17×	78.9%

Table 3: Initial medium-input scalability medians on GHC with $P = 0.1$, five iterations, and batch size one. Speedups use this implementation’s one-thread baseline; efficiency uses computation speedup. Total time is initialization plus computation. All outputs passed validation.

Eight-thread computation speedup was $6.31\times$, with approximately 79% efficiency. The gains are substantial but diminish as threads are added.

We do not expect perfect speedup. Initial route placement, occupancy updates, random-route choices, and the final per-wire reduction remain se-

quential. Many searches also enter separate parallel regions and barriers. Small searches offer limited candidate-level parallelism, and occupancy reads can be constrained by cache behavior or memory bandwidth. Candidate generation is now parallel on the parallel path, so sequential full-vector generation is no longer a remaining bottleneck in that path. Profiling is needed to distinguish the remaining synchronization, memory, and sequential-work costs.

We observed improving performance, not a drop-off, from one through eight threads in the initial snapshot. Efficiency declined as thread count increased. At higher counts, limited candidates per wire, synchronization overhead, and memory-system pressure may flatten or reverse scaling, but this remains a hypothesis to test in the PSC experiments.

5 Planned Across-Wires Parallelism

The across-wires strategy is more complicated because different threads make route decisions that update the shared occupancy matrix. Our initial plan is to follow the assignment’s batch model: a worker obtains a batch of wires, processes the wires in that batch sequentially, and then updates the occupancy matrix for the chosen routes. We will vary the batch size to study the tradeoff between synchronization overhead, staleness, load balance, and final routing quality.

Potential synchronization strategies include a single global lock, row-level locks, and tile-level locks. We expect the simplest correct version to use coarser synchronization first, then refine the locking granularity only if measurements show that update contention is a bottleneck.

6 Optimization Hypotheses

The main performance themes we plan to evaluate are workload balance, scheduling overhead, locality, and contention. Within-wires parallelism exposes candidate routes as independent tasks; this should work best for wires with large bounding boxes and many possible routes. Across-wires parallelism exposes wires or batches of wires as tasks; this may require dynamic scheduling or longest-task-first ordering to avoid stragglers.

We will also consider locality-aware across-wires batching as a later experiment. For example, wires could be grouped by bounding-box midpoint or quadrant so that threads tend to work on nearby occupancy cells. This may improve cache locality, but it may also increase contention because

nearby wires are more likely to overlap. We will only pursue this direction if cache miss or synchronization measurements suggest that the extra work is justified.

7 Experimental Plan

For the GHC experiments, we will collect results for 1, 2, 4, and 8 threads for both within-wires and across-wires implementations. We will report total time, computation time, speedup, cache misses, maximum occupancy, and total routing cost. After the GHC implementation is stable, we will run the required PSC scaling experiments for 1, 2, 4, 8, 16, 32, 64, and 128 threads.